

# Store Procedures

Marcos Rodrigo Momo

Blumenau, Setembro 2026.

# Introdução a Stored Procedure

- As *Stored Procedures* (procedimentos armazenados) do MySQL são usados para agrupar uma ou mais instruções SQL em unidades lógicas.
- As *Stored Procedures* são armazenadas como objetos nomeados no servidor de banco de dados do MySQL.
- Pode ser uma forma de minimizar o processamento do lado da aplicação, principalmente web e minimizar tráfego de códigos pesados pela rede.

# Desvantagens Store Procedure

- O MySQL não fornece boas ferramentas de desenvolvimento e depuração, portanto, é relativamente difícil depurar SPs.
- A eficiência da linguagem SQL em si não é tão alta quanto a da linguagem de programação de aplicativos e é relativamente básica.
- As SPs também podem aumentar a complexidade da implantação do aplicativo. Não apenas o código do aplicativo e as tabelas de banco de dados precisam ser implantados, mas também as SPs precisam ser implantados, e se precisar migrar de sistema gerenciador de banco de dados,
- Você terá que reescrever parte de todos os códigos SQL das SPs.
- No caso do MYSQL, o plano de execução de cada SP conectado é armazenado no cache (memória ram) de forma independente.
- Se muitas conexões chamarem a mesma SP, o armazenamento em cache repetido causará um desperdício de recursos (memória ram).
- O código SQL da SP é difícil de interpretar. É difícil analisar um procedimento com baixa performance por exemplo, quando existe um procedimento com muitas instruções SQL dentro.

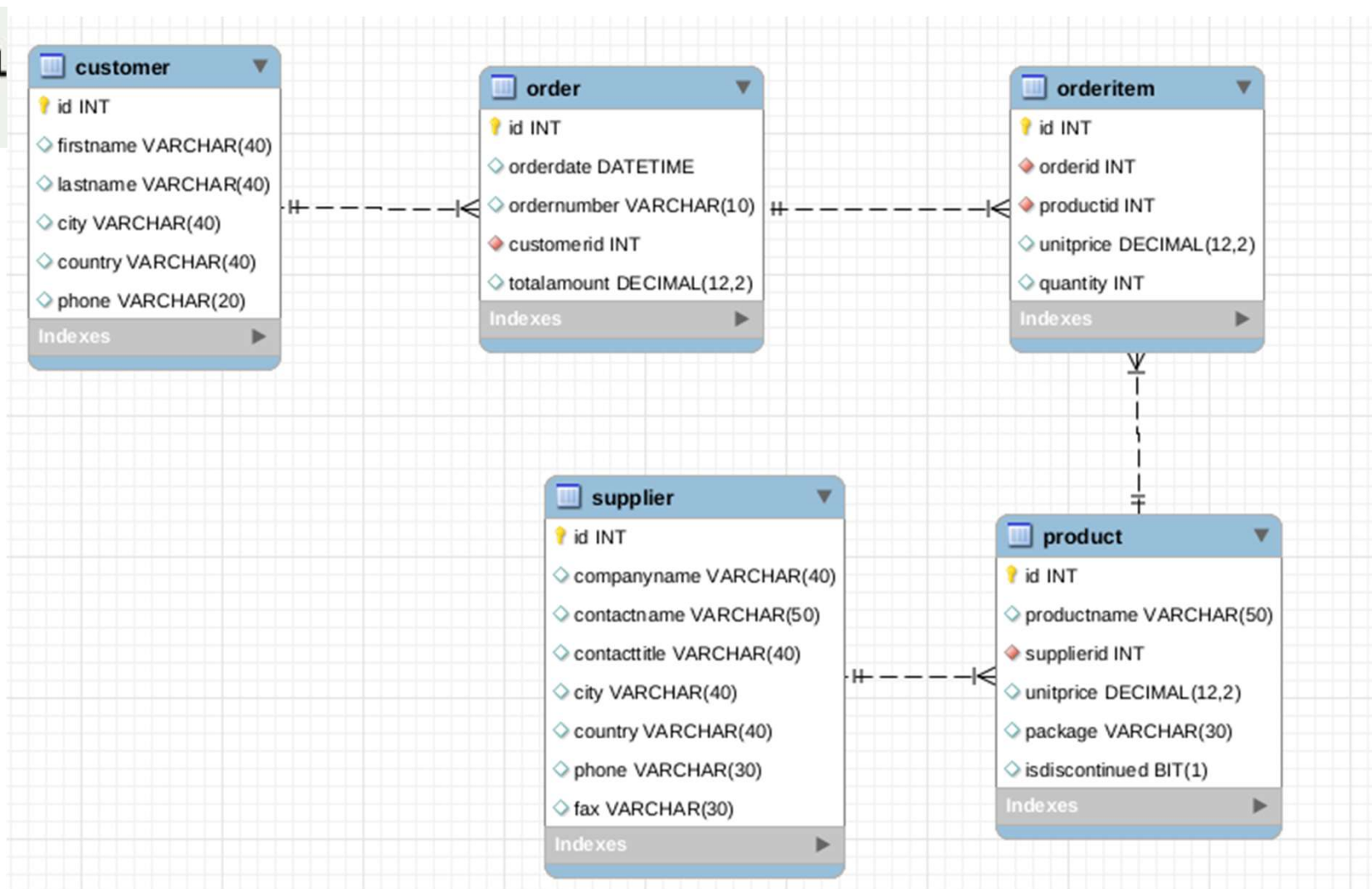
Evite SP muito grandes.

# Vantagens Store Procedure

- Ela roda diretamente na camada do banco de dados, de forma a reduzir a ocupação da largura de banda da rede e o atraso na execução da tarefa de consulta.
- Ela melhora a capacidade de reutilização e manutenção do código, agrega regras de negócios e melhora a segurança (posso por exemplo dar acesso a sp e não tabela diretamente).
- O servidor armazenará em cache (memória ram) o plano que o otimizador de query usou de acordo com parâmetros usados e pode reaproveitá-lo na execução das *Stored Procedures*.
- O que pode reduzir a carga de execução repetida.
- Não perde tempo em a cada chamada ter que verificar a sintaxe dos comandos internos e não tem que compilar novamente.

# Vamos utilizar os scripts do Moodle

- Scripts DDL e DML aula procedure.sql
- Gerar um Script chamado de procedure.sql



# Práticas

-- RODAR

```
SELECT  Id
          ,ProductName
          ,SupplierId
          ,UnitPrice
          ,Package
          ,IsDiscontinued
FROM CLIENTE2.Product
ORDER BY ProductName DESC;
```

-- CRIANDO PRIMEIRA STORED PROCEDURE

DELIMITER \$\$

```
CREATE PROCEDURE ProductList()
BEGIN
```

```
    SELECT Id
            ,ProductName
            ,SupplierId
            ,UnitPrice
            ,Package
            ,IsDiscontinued
    FROM CLIENTE2.Product
    ORDER BY ProductName DESC;
END$$
```

DELIMITER ;

-- EXECUTANDO A STORED PROCEDURE CRIADA

```
call ProductList();
```

# Práticas

-- VERIFICAR A STORED PROCEDURE CRIADA NO WORKBENCH E VIA SCRIPTS ABAIXO:

SHOW PROCEDURE STATUS;

-- OU

use cliente2;

```
select routine_name, routine_type, definer, created, security_type, SQL_Data_Access  
from information_schema.routines  
where routine_type='PROCEDURE' and routine_schema='cliente2';
```

# Práticas

```
-- MODIFICANDO A STORED PROCEDURE
|
-- NO SQL SERVER PODERIA USAR O ALTER PROCEDURE MAS NO MYSQL 8 E VRS ANTERIORES PRECISA DELETAR E CRIAR NOVAMENTE

DROP PROCEDURE IF EXISTS ProductList;

DELIMITER $$

CREATE PROCEDURE ProductList()
BEGIN
    SELECT Id
        ,ProductName
        ,SupplierId
        ,UnitPrice
    FROM CLIENTE2.Product
    ORDER BY ProductName ASC
    LIMIT 3;
END$$

DELIMITER ;

-- EXECUTE NOVAMENTE E VEJA O RESULTADO

CALL ProductList();
```

# Práticas

```
-- DELETANDO STORED PROCEDURE  
DROP PROCEDURE ProductList;
```

# Prática

•Store procedures, aceita três tipos de parâmetros, podemos passar parâmetros do tipo:

- IN: parâmetros de entrada
- OUT: parâmetros de saída
- INOUT: parâmetro de entrada e saída

# Parâmetro IN

```
-- IN Parameters
```

```
DROP PROCEDURE ProductList;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE ProductList(IN max_listprice DECIMAL(12,2))
```

```
BEGIN
```

```
    SELECT Id
```

```
        ,ProductName
```

```
        ,SupplierId
```

```
        ,UnitPrice
```

```
FROM CLIENTE2.Product
```

```
WHERE
```

```
    UnitPrice <= max_listprice
```

```
ORDER BY ProductName ASC;
```

```
END$$
```

```
DELIMITER ;
```

```
-- vamos chamar a proc passando parametro de valor maximo de preco para
```

```
-- a proc trazer produtos mais baratos
```

```
CALL ProductList(21.00);
```

## Analise

```
-- Podemos rodar o select abaixo para ver a lista completo
```

```
SELECT Id
```

```
        ,ProductName
```

```
        ,SupplierId
```

```
        ,UnitPrice
```

```
FROM CLIENTE2.Product;
```

## Dois parâmetros IN

-- Passando mais de 1 Parametro para a Stored Procedure

**DROP** PROCEDURE IF **EXISTS** ProductList;

DELIMITER \$\$

**CREATE** PROCEDURE ProductList (**IN** min\_listprice **DECIMAL**(12,2),  
**IN** max\_listprice **DECIMAL**(12,2))

**BEGIN**

**SELECT** Id  
    ,ProductName  
    ,SupplierId  
    ,UnitPrice  
**FROM** CLIENTE2.Product  
**WHERE**  
    UnitPrice <= max\_listprice **and**  
    UnitPrice >= min\_listprice  
**ORDER BY** ProductName **ASC**;  
**END\$\$**

DELIMITER ;

Atenção para a ordem dos parâmetros

-- Agora chame a PROCEDURE passando os 2 parametros

**call** ProductList (10, 200); -- a ordem da passagem dos parametros é essencial.

02/09/2026

**call** ProductList (12, 14);

```
DELIMITER $$
```

```
CREATE PROCEDURE ProductList (IN min_listprice DECIMAL(12,2),  
    IN max_listprice DECIMAL(12,2),  
    IN ProductNamePASSADA VARCHAR(50))  
BEGIN  
    SELECT Id  
        ,ProductName  
        ,SupplierId  
        ,UnitPrice  
    FROM CLIENTE2.Product  
    WHERE  
        UnitPrice <= max_listprice and  
        UnitPrice >= min_listprice and  
        productname LIKE CONCAT ('%' , productnamePASSADA , '%')  
    ORDER BY UnitPrice DESC;  
END$$
```

```
DELIMITER ;
```

analise saída

-- VAMOS EXECUTAR PASSANDO OS 3 PARAMENTROS

```
CALL ProductList (12, 15, 'hok');
```

## Três parâmetros IN

# Parâmetros opcionais

- Ao executar Stored Procedure ProductList, você deve passar todos os três argumentos correspondentes aos três parâmetros.
- Você possa ignorar os parâmetros com os valores padrão.
- Solução de contorno!!!!

# Parâmetros opcionais

```
DROP PROCEDURE IF EXISTS ProductList;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE ProductList (IN min_listprice DECIMAL(12,2),  
    IN max_listprice DECIMAL(12,2),  
    IN productnamePASSADA VARCHAR(50))
```

```
BEGIN
```

```
IF min_listprice IS NULL THEN  
    set min_listprice=0.00;  
END IF;
```

```
if max_listprice IS NULL THEN  
    set max_listprice=999999999.99;  
END IF;
```

```
SELECT Id  
        ,ProductName  
        ,UnitPrice  
FROM CLIENTE2.Product  
WHERE  
        UnitPrice >= min_listprice and  
        UnitPrice <= max_listprice and  
        productname LIKE CONCAT ('%' , productnamePASSADA , '%')  
ORDER BY  
        id;  
END$$
```

```
DELIMITER ;
```

Faça os testes e analise saídas

```
-- VAMOS AGORA CHAMAR A PROCEDURE PASSANDO PARAMETROS
```

```
CALL ProductList (12, 15, 'a');
```

```
CALL ProductList (NULL, NULL, 'a');
```

```
CALL ProductList (70, NULL, 'a');
```

# Trabalhando com variáveis

```
SET @orderdate = 2012; -- COLOCANDO VALORES NAS VARIÁVEIS.
```

```
SELECT Id  
       ,OrderDate  
       ,OrderNumber  
       ,CustomerId  
       TotalAmount  
FROM CLIENTE2.Order  
where year(OrderDate) = @orderdate ;
```

```
-- Armazenando o resultado de uma consulta em uma variavel
```

```
SET @supplier_count = (  
    SELECT  
        COUNT(*)  
    FROM  
        supplier  
);
```

```
SELECT @supplier_count ; -- demonstrando o valor guardado na variavel @supplier_count
```

# Trabalhando com variáveis

```
-- Armazenando valores em variaveis  
-- ATENCAO PARA USO DO := QUANDO TIVER MAIS DE UM CAMPO  
-- NO SELECT RECEBENDO DADOS
```

```
SELECT      @productnameVAR := ProductName  
            , @listpriceVAR  := UnitPrice  
FROM CLIENTE2.Product  
WHERE  
      id = 1;
```

```
SELECT @productnameVAR AS NomeProduto;  
SELECT @listpriceVAR   as PrecoUnitario;
```

# Parâmetro OUT

-- A seguinte SP (Stored Procedure) retorna a quantidade de produtos existente

```
DROP PROCEDURE IF EXISTS totalprodutos;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE totalprodutos (  
    OUT count_produto int  
)
```

```
BEGIN
```

```
    SELECT
```

```
        count(id)
```

```
        INTO count_produto
```

```
    FROM
```

```
        product;
```

```
END$$
```

```
DELIMITER ;
```

analise saída

-- CHAMANDO A SP

```
CALL totalprodutos(@totalprodutos);  
SELECT @totalprodutos;
```

# Parâmetros IN OUT

```
-- A seguinte SP (Stored Procedure) retorna o nome do  
-- produto com passagem de um código de um produto específico:
```

```
DROP PROCEDURE IF EXISTS Acharproduto;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE Acharproduto (  
    IN produto_ID INT ,  
    OUT nome_produto VARCHAR(50)  
)
```

```
BEGIN
```

```
    SELECT
```

```
        productname
```

```
    INTO nome_produto
```

```
FROM
```

```
    product
```

```
WHERE
```

```
    ID = produto_ID;
```

```
END$$
```

```
DELIMITER ;
```

02/09/2026

```
-- CHAMANDO A SP, passando parametro precounitario = 2
```

```
SET @produto = 2;
```

```
CALL Acharproduto(@produto, @nome_produto);
```

```
SELECT @nome_produto;
```

analise saída

# Laço condicional

analise saída

```
-- ELSE IF EXEMPLO

DELIMITER $$

DROP PROCEDURE IF EXISTS SOMA ;

CREATE PROCEDURE SOMA ()

BEGIN

    SELECT
        @vendas := SUM(unitprice * quantity) -- atencao para usar :=
    FROM
        orderitem i
    INNER JOIN cliente2.Order oo ON oo.id = i.OrderId
    WHERE
        YEAR(oo.orderdate) = 2012;

    SELECT @vendas;

    IF @vendas < 100 then
        SELECT @VENDAS AS 'Vendas de 2012 estao MENORES que 100';
    ELSEIF @vendas < 500 THEN
        SELECT @VENDAS AS 'Vendas de 2012 estao MENORES que 500';
    ELSE
        SELECT @VENDAS AS 'Vendas de 2012 estao IGUAL OU MAIORES que 500';
    END IF;

END$$

DELIMITER ;
```

```
-- VAMOS CHAMAS A PROC

CALL SOMA();
```

# Laço repetição

```
-- WHILE EXEMPLO
```

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS testawhile ;
```

```
CREATE PROCEDURE testawhile ()
```

```
BEGIN
```

```
SET @qt = 0;
```

```
WHILE @qt <= 5 DO
```

```
    SELECT @qt;
```

```
    SET @qt = @qt + 1;
```

```
END WHILE;
```

```
END$$
```

```
DELIMITER ;
```

analise saída

```
-- VAMOS CHAMAS A PROC
```

```
CALL testawhile ();
```

```
-- WHILE EXEMPLO SAINDO DO LOOP COM COMANDO LEAVE.
== NO SQL SERVER PARA SAIR DO LOOP IRIAMOS USAR O COMANDO BREAK

DELIMITER $$

DROP PROCEDURE IF EXISTS testawhile ;

CREATE PROCEDURE testawhile ()

BEGIN

SET @qt = 0;

myloop: WHILE @qt <= 5
DO
    SELECT @qt;
    SET @qt = @qt + 1;
    if @qt = 3 then
        LEAVE myloop;
    end if;
END WHILE myloop;

END$$

DELIMITER ;
```

# Laço repetição

analise saída

```
-- CHAMAR A PROCEDURE

CALL testawhile();
```